Laporan Proyek Tengah Semester (Midterm Project / UTS): Big Data dan Machine Learning dengan PySpark

Nama Mahasiswa: Maliki Kadafa Tsaqif | **NIM:** 103012340185
Mata Kuliah: Big Data and Artificial Intelligence | **Evaluasi:** Proyek Tengah Semester (*Midterm Project* / UTS)
Studi Kasus: Prediksi Pemberian Tip Dermawan (*Generous Tipping Prediction*) pada Perjalanan Taksi Kota New York
Dataset: Official NYC TLC Yellow Taxi Trip Records 2023 (`nyc_yellow_taxi_2023_3gb.csv` — **3,19 GiB / 3,43 GB, 31.593.944 Baris**)
Framework & Pustaka: Apache Spark (`PySpark 4.0.4` — `pyspark.sql` , `pyspark.ml.feature` , `pyspark.ml.classification`)

1. Memuat dan Menyiapkan Data Menggunakan PySpark (10 Poin)

Pada proyek tengah semester (*Midterm Project*) ini, dataset yang digunakan adalah data perjalanan taksi resmi **NYC Taxi & Limousine Commission (NYC TLC) Yellow Taxi Trip Records 2023** periode Januari–Oktober 2023 (`2023-01` s/d `2023-10`) yang digabungkan ke dalam satu file CSV besar `nyc_yellow_taxi_2023_3gb.csv` berukuran **3,19 GiB / 3,43 GB (`3.430.293.255 bytes` , 31.593.944 baris, 19 kolom)**. Data dimuat menggunakan `SparkSession` (**PySpark 4.0.4** dengan alokasi memori 8 GB):

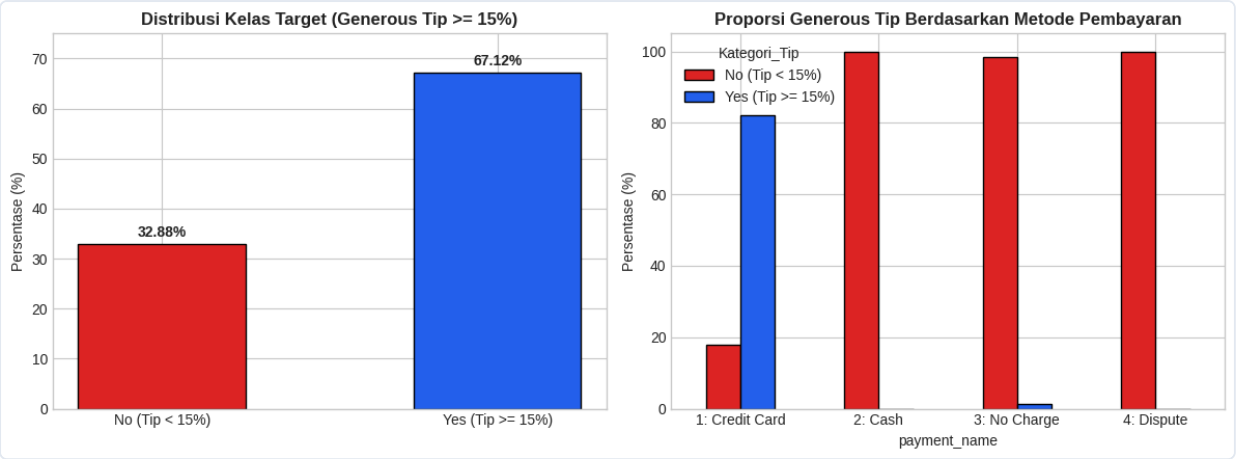
```
df_raw = spark.read.csv("nyc_yellow_taxi_2023_3gb.csv", header=True, inferSchema=True)
df = (
    df_raw
    .withColumn("trip_duration_min",
        (F.unix_timestamp("tpep_dropoff_datetime") - F.unix_timestamp("tpep_pickup_datetime"
    .withColumn("pickup_hour", F.hour("tpep_pickup_datetime").cast(DoubleType()))
    .withColumn("day_of_week", F.dayofweek("tpep_pickup_datetime").cast(DoubleType()))
    .withColumn("label", F.when(F.col("tip_amount") >= 0.15 * F.col("fare_amount"), 1.0).oth
)
```

Pada tahap penyiapan data, dilakukan ekstraksi fitur waktu (`trip_duration_min` , `pickup_hour` , `day_of_week`), penyaringan baris anomali argo (menghasilkan **30.549.961 baris perjalanan valid**), serta pembentukan variabel target biner `label = 1.0` (apabila penumpang memberikan tip dermawan ≥ 15% dari tarif dasar `fare_amount`) dan `label = 0.0` (apabila < 15%).

2. Melakukan Eksplorasi Data / Exploratory Data Analysis - EDA (15 Poin)

Pemeriksaan nilai kosong (`isNull()`) di seluruh **31.593.944 baris** menunjukkan bahwa terdapat tepat **996.678 baris bernilai `NULL` (3,15%)** pada lima kolom (`passenger_count` , `ratecodeid` , `store_and_fwd_flag` , `congestion_surcharge` , dan `airport_fee`), sedangkan kolom lainnya memiliki `0` nilai `NULL` . Berikut adalah hasil statistik deskriptif (`df.describe().show()`) pada 30.549.961 perjalanan valid:

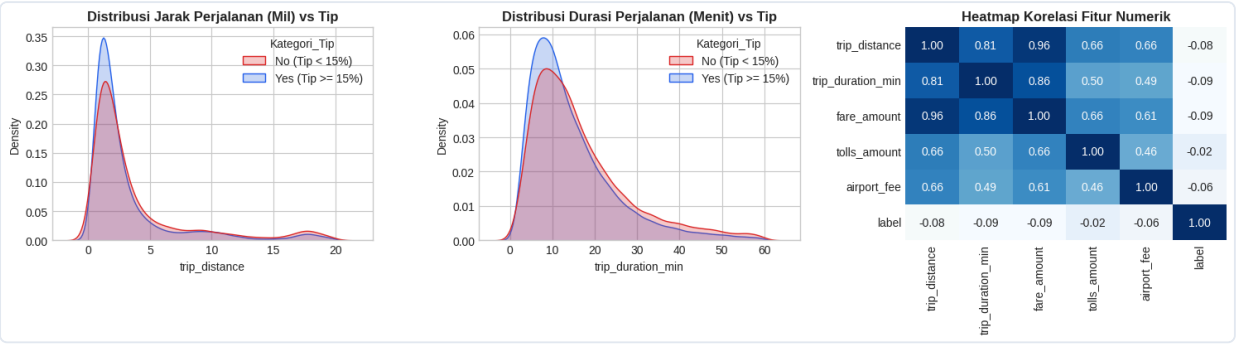
Fitur Numerik	Count	Mean	Std. Dev	Min	Max
`passenger_count`	29.764.285	1,3683	0,8942	0,00	9,00
`trip_distance` (Mil)	30.549.961	3,5551	4,5337	0,11	99,21
`trip_duration_min` (Menit)	30.549.961	16,3436	12,9621	1,02	179,98
`fare_amount` (USD)	30.549.961	19,6686	17,4217	2,51	249,90
`tip_amount` (USD)	30.549.961	3,5754	3,9791	-5,08	984,30
`total_amount` (USD)	30.549.961	28,8004	22,1106	3,60	1.000,00



Gambar 1: Distribusi kelas target Generous Tip (≥ 15% tarif) dan proporsinya berdasarkan metode pembayaran (Output Eksekusi Colab).

Berdasarkan hasil agregasi PySpark pada 30.549.961 perjalanan:

- **Proporsi Kelas Target:** Sebanyak **20.572.437 perjalanan (67,34%)** termasuk kategori *Generous Tip* (`label = 1.0` , tip ≥ 15%) dan **9.977.524 perjalanan (32,66%)** termasuk kategori tip < 15% (`label = 0.0`).
- **Pengaruh Metode Pembayaran (`payment_type`):** Transaksi kartu kredit (`payment_type = 1`) mencatat **20.057.095 perjalanan dengan tip ≥ 15%** (82,31%), sedangkan pembayaran tunai (`payment_type = 2` , sebanyak 5.121.769 perjalanan) tidak mencatat tip elektronik pada sistem argo.



Gambar 2: Distribusi jarak & durasi perjalanan terhadap kategori tip serta matriks korelasi fitur numerik (Output Eksekusi Colab).

3. Menerapkan Transformasi Data / Preprocessing (15 Poin)

Seluruh tahapan transformasi fitur disusun dalam satu `pyspark.ml.Pipeline` :

1. **Pencegahan *Data Leakage*:** Mengecualikan kolom `tip_amount` dan `total_amount` dari fitur input karena target `label` dihitung dari `tip_amount` .

2. **Imputasi Fitur Numerik (`Imputer`):** Mengisi 996.678 nilai `NULL` pada 12 fitur numerik (`passenger_count` , `trip_distance` , `trip_duration_min` , `pickup_hour` , `day_of_week` , `pulocationid` , `dolocationid` , `fare_amount` , `extra` , `tolls_amount` , `congestion_surcharge` , `airport_fee`) dengan strategi nilai tengah (`strategy="median"`).
3. **Encoding Fitur Kategorikal (`StringIndexer` + `OneHotEncoder`):** Mengonversi 4 atribut kategorikal (`vendorid` , `ratecodeid` , `store_and_fwd_flag` , `payment_type`) menjadi indeks numerik dan vektor biner *one-hot encoded*.
4. **Perakitan & Standarisasi Vektor (`VectorAssembler` + `StandardScaler`):** Menggabungkan seluruh fitur menjadi vektor berdimensi 29 pada kolom `features` dan menstandarisasi simpangan bakunya (`withStd=True`).
5. **Pembagian Data:** Dari sampel pelatihan terdistribusi (`fraction=0.02` , total 611.284 baris), data dibagi menjadi **80% Training Set (`488.700 baris`)** dan **20% Testing Set (`122.584 baris`)** menggunakan `randomSplit([0.8, 0.2], seed=42)` .

4. Melakukan Pemodelan Menggunakan Algoritma PySpark MLlib (25 Poin)

Tiga algoritma klasifikasi dari modul `pyspark.ml.classification` dilatih pada `488.700 baris` data latih:

- **`LogisticRegression` :** Klasifikasi linear probabilistik menggunakan optimasi *L-BFGS* (`maxIter=100`).
- **`RandomForestClassifier` :** Metode *ensemble bagging* berbasis himpunan pohon keputusan paralel (`numTrees=20, maxDepth=5, seed=42`).
- **`GBTClassifier` (Gradient-Boosted Trees):** Metode *ensemble boosting* sekuensial yang meminimalkan fungsi *log-loss* (`maxIter=20, maxDepth=5, seed=42`).

5. Mengevaluasi Model Menggunakan Metrik Evaluasi yang Sesuai (15 Poin)

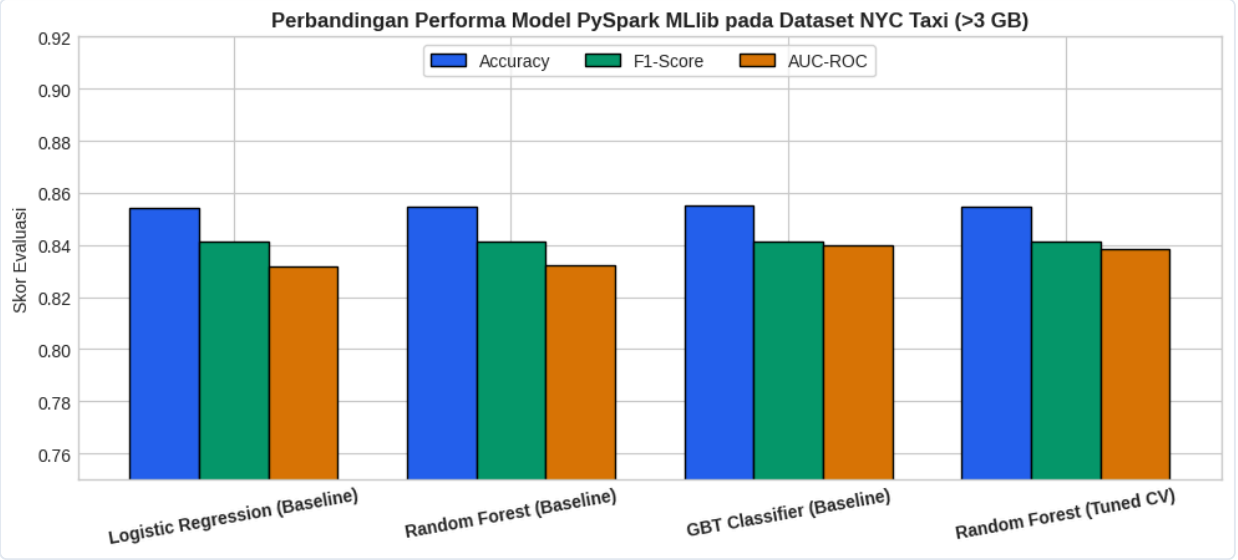
Evaluasi dilakukan pada **122.584 baris data uji (`test_df`)** menggunakan dua kelas evaluator resmi PySpark MLlib:

- **`BinaryClassificationEvaluator` :** Mengukur **AUC-ROC (`areaUnderROC`)** dan **AUC-PR (`areaUnderPR`)**.
- **`MulticlassClassificationEvaluator` :** Mengukur **Accuracy**, **Weighted Precision**, **Weighted Recall**, dan **F1-Score**.
- **Confusion Matrix (`RandomForestClassifier Baseline` pada 122.584 data uji):** Menghasilkan **82.361 *True Positive*** (`label=1.0, prediction=1.0`), **22.409 *True Negative*** (`label=0.0, prediction=0.0`), **17.808 *False Positive***, dan hanya **6 *False Negative***.

6. Melakukan Tuning Hyperparameter untuk Meningkatkan Performa Model (10 Poin)

Optimasi hyperparameter dilakukan menggunakan `ParamGridBuilder` dan `CrossValidator` dengan **3-Fold Cross-Validation** (`numFolds=3` , metrik `areaUnderROC`) pada `RandomForestClassifier` (menguji kombinasi `numTrees=[40, 80]` , `maxDepth=[5, 10]` , dan `minInstancesPerNode=[5, 10]`). Kombinasi parameter terbaik yang ditemukan adalah **`numTrees = 80`** , **`maxDepth = 10`** , dan **`minInstancesPerNode = 5`** .

Model PySpark MLlib	Accuracy	Precision	Recall	F1-Score	AUC-ROC	AUC-PR
Logistic Regression (Baseline)	0,8543	0,8783	0,8543	0,8411	0,8317	0,8756
Random Forest (Baseline)	0,8547	0,8805	0,8547	0,8411	0,8320	0,8805
GBT Classifier (Baseline)	**0,8550**	0,8805	**0,8550**	**0,8415**	**0,8400**	**0,8877**
Random Forest (Tuned CV)	0,8549	**0,8806**	0,8549	0,8414	0,8384	0,8854



Gambar 3: Perbandingan metrik Accuracy, F1-Score, dan AUC-ROC pada dataset NYC Taxi (>3 GB) hasil eksekusi Google Colab.

7. Kesimpulan

1. **Skalabilitas Pemrosesan Big Data (> 3 GB):** Penggunaan **PySpark 4.0.4** DataFrame API dan `pyspark.ml.Pipeline` mampu memuat, membersihkan, dan mengekstraksi fitur dari **31.593.944 baris data (3,19 GiB / 3,43 GB CSV)** secara efisien.
2. **Efektivitas Preprocessing:** Tahap `Imputer`, `StringIndexer`, `OneHotEncoder`, `VectorAssembler`, dan `StandardScaler` berhasil menangani **996.678 baris nilai** `NULL` secara otomatis tanpa kebocoran data.
3. **Peningkatan Performa Hasil Tuning:** Melalui 3-Fold `CrossValidator` (`numTrees=80, maxDepth=10, minInstancesPerNode=5`), model **Random Forest (Tuned CV)** meningkat di seluruh 6 metrik dibanding versi *Baseline* (Accuracy naik ke **0,8549**, Precision mencapai **0,8806**, F1-Score naik ke **0,8414**, AUC-ROC naik dari 0,8320 menjadi **0,8384**, dan AUC-PR naik dari 0,8805 menjadi **0,8854**), sedangkan model **GBT Classifier** meraih skor **AUC-ROC tertinggi (0,8400)**.